

# 企业财务数据分析与造假识别

## 摘要

财务数据分析是对企业的财务状况和经营成果进行评估和预测的过程。它对企业的生产经营管理和外部投资、贷款、赊销等决策起到重要的作用，同时也为经济管理部门提供了了解行业经济运行状况的依据，以促进经济的合理配置和科学调控。然而，财务造假对企业、投资者和整个市场产生严重的威胁。因此，准确甄别财务造假企业、帮助投资者识别风险以及促进企业的诚信经营对于营造良性的市场竞争环境具有重要的现实意义。只有确保财务数据的准确性和可信度，才能保障企业和市场的可持续发展。本文将围绕聚焦于企业数据分析与造假识别，基于机器学习集成算法进行建模，识别企业公司的财务造假行为，主要研究内容和成果包括以下三部分：

**针对问题一：**首先对于所给原始财务数据集进行数据预处理，对其中的缺失、重复、异常值进行相应的剔除或赋值等操作。同时对已有数据进行整合，提取相关数据进行合并，以便对财务数据进行分析，并根据已有数据计算利润率和资产负债率等企业财务数据指标，以进行更好的企业利润预测以及财务造假识别。

**针对问题二：**为进行行业与企业营收的分析，对已经初步处理的数据，分别统计不同行业利润总额均值，绘制各大行业的利润率均值变化，并绘制相应对比分析图；对利润率最高的行业，分别统计其不同细类的利润率，及其细类下企业利润率的排序，并绘制对应柱状图表；同时选取“T1”企业，对其进行营业总成本分析和经营情况的分析，并绘制对应的饼图和相应柱状折线组合图；最终将所有图表数据进行整合为一个行业与企业营业数据分析大屏。

**针对问题三：**通过计算各个指标与利润总额的相关性选取“营业收入”、“业务费用”、“营业成本”、“营业税金及附加”、“资产减值损失”这五个指标，采用BP神经网络算法构建预测模型，并对企业的利润进行预测，机器学习集成学习算法中XGBoost分类预测算法，利用“流动比率”、“资产负债率”、“存货周转率”、“资产报酬率”、“应收账款周转率”这五个关键因子，挖掘财务造假的识别特征，识别出其中唯一一个涉嫌财务造假的企业。

关键词：XGBoost 分类预测算法、BP 神经网络模型

# 目录

|                                       |    |
|---------------------------------------|----|
| 摘要.....                               | 1  |
| 一、问题分析.....                           | 4  |
| 二、数据预处理.....                          | 4  |
| 2.1 缺失值处理.....                        | 4  |
| 2.1.1 缺失情况说明.....                     | 4  |
| 2.1.2 缺失数据处理过程.....                   | 4  |
| 三、任务一：财务数据处理.....                     | 6  |
| 任务 1.1.....                           | 6  |
| 任务 1.2.....                           | 8  |
| 任务 1.3.....                           | 10 |
| 任务 1.4.....                           | 12 |
| 任务 1.5.....                           | 14 |
| 任务 1.6.....                           | 15 |
| 四、任务二：财务数据指标分析及可视化.....               | 17 |
| 任务 2.1 根据文件绘制“行业营业利润对比分析”图.....       | 17 |
| 2.1.1 实现思路.....                       | 17 |
| 2.1.2 代码运行及结果.....                    | 18 |
| 柱状图.....                              | 18 |
| 2.1.3 结果分析.....                       | 23 |
| 任务 2.2 根据文件绘制该行业大类相关的“行业企业营收分析”图..... | 25 |
| 2.2.1 实现思路.....                       | 25 |
| 2.2.2 代码运行及结果.....                    | 26 |
| 2.2.3 结果分析.....                       | 37 |
| 2019 年该行业各细 类利润率对比图.....              | 37 |
| 1. 排名前 3 细类的利润率对比：.....               | 37 |
| 2. 行业细类利润率差异：.....                    | 37 |
| 3. 决策参考：.....                         | 37 |
| 1. 行业利润率排名：.....                      | 37 |
| 2. 企业利润率对比：.....                      | 38 |
| 3. 企业标识：.....                         | 38 |
| 4. 决策参考：.....                         | 38 |
| 1. 构成比例：.....                         | 38 |
| 2. 主要支出项目：.....                       | 38 |
| 3. 财务稳健性：.....                        | 38 |
| 4. 决策参考：.....                         | 38 |
| 1. 营业总成本和营业总收入柱状图：.....               | 39 |
| 2. 利润率和资产负债率的数据输出：.....               | 39 |
| 任务 2.3 绘制“行业与企业营业数据分析”的可视化大屏.....     | 39 |
| 2.3.1 实现思路.....                       | 39 |
| 2.3.2 代码运行及结果.....                    | 39 |
| 2.3.3 结果分析.....                       | 42 |
| 五、任务三：企业利润预测及财务造假识别.....              | 43 |
| 任务 3.1.....                           | 43 |
| 3.1.1 实现思路.....                       | 43 |
| 3.1.2 实现过程.....                       | 43 |
| 3.1.3 计算结果.....                       | 44 |
| 任务 3.2.....                           | 45 |
| 3.2.1 预测模型构建.....                     | 45 |
| 3.2.2 实现过程.....                       | 46 |
| 1. 数据准备.....                          | 46 |
| 2. 神经网络设计.....                        | 46 |

|                          |    |
|--------------------------|----|
| 3. 模型训练 .....            | 46 |
| 4. 模型评估 .....            | 47 |
| 5. 结果解释和展示 .....         | 47 |
| 6. 模型优化和更新 .....         | 47 |
| 3.2.3 预测结果.....          | 47 |
| 任务 3.3 .....             | 48 |
| 3.3.1 财务造假识别方法.....      | 48 |
| 3.3.2 指标的初步选取与体系构建 ..... | 49 |
| 3.3.3 数据处理.....          | 50 |
| 3.3.4 模型设计.....          | 51 |
| 3.3.5 解决过程.....          | 53 |
| 六、参考文献 .....             | 55 |

## 一、问题分析

1. 财务数据预处理。
2. 财务数据指标分析及可视化。
3. 建立企业利润预测模型，预测企业营收情况；并根据财务报表数据的特点，筛查财务造假的企业。

## 二、数据预处理

为了能更有效的处理数据信息，需要先对数据进行预处理，使原始数据变得规则、统一。通过使用 `data.describe()` 快速了解数据集的分布和基本统计信息，便于我们得到关于数据中的趋势、离散程度和异常值等方面的一些信息。

### 2.1 缺失值处理

#### 2.1.1 缺失情况说明

本次数据分析过程中，缺失数据将被进一步划分为两个部分，一类是数据为空值的情况，一类是数据为零的情况。

针对不同的数据缺失情况，本次分析将会采用不同的处理方式：

1、针对数值为 0 的情况，需要进行实际的分析，回归到原始数据中去，判断该数据为 0 时是否具有实际意义。如果没有就将其作为缺失值做删除处理。

2、针对数据为空值的情况，如果该特征数据缺失情况低于 10%，则结合该特征的重要性进行综合判断。如果字段重要性较低，则考虑直接删除，如果字段重要性较高，则进行插值法或者采用数据均值进行填补。

#### 2.1.2 缺失数据处理过程

使用 `data.info()` 函数去显示数据集的摘要信息，包括数据类型、列名、非空值的数量和内存使用量等。这些信息对于理解数据集的大小、缺失值、处理数据类型等非常有用，能够帮助我们更有效地进行数据分析和处理。

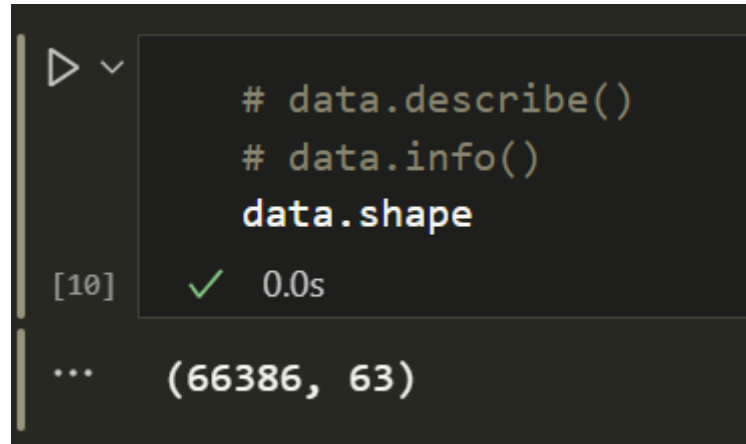
```
> ✓ # data.describe()
data.info()
# data.shape()

[8] ✓ 0.0s

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 66386 entries, 0 to 66385
Data columns (total 63 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Stkcd                  66386 non-null  int64
1   Accper                 66386 non-null  object
2   Typrep                 66386 non-null  object
3   B001100000             64453 non-null  float64
4   B001101000             63093 non-null  float64
5   Bbd1102000             2285 non-null   float64
6   Bbd1102101             1997 non-null   float64
7   Bbd1102203             1513 non-null   float64
8   B0i1103000             187 non-null    float64
9   B0i1103101             89 non-null     float64
10  B0i1103111             51 non-null     float64
11  B0i1103203             79 non-null     float64
12  B0i1103303             79 non-null     float64
13  B0d1104000             1738 non-null   float64
14  B0d1104101             629 non-null    float64
15  B0d1104201             624 non-null    float64
16  B0d1104301             511 non-null    float64
17  B0d1104401             1075 non-null   float64
18  B0d1104501             615 non-null    float64
19  B0f1105000             1299 non-null   float64
...
61  B006000101             33342 non-null  float64
62  B006000102             27343 non-null  float64
dtypes: float64(60), int64(1), object(2)
memory usage: 31.9+ MB
```

Output is truncated. View as raw/html/JSON or open in a text editor.

通过 `data.shape` 查询数据的形状，了解数据集在不同维度上的大小或长度，对于数据预处理、模型构建和结果解释非常重要。它可以帮助确定数据集的结构，从而选择恰当的分析方法或模型。



```
# data.describe()
# data.info()
data.shape

[10] ✓ 0.0s

... (66386, 63)
```

### 三、任务一：财务数据处理

#### 3.1 任务 1.1

首先要导入 pandas 库，以便可以使用其中的函数和方法对数据进行处理和分析。然后使用 `pd.read_csv()` 函数读取目标 CSV 文件，并将其存储在一个 DataFrame 中，例如 `lr_data`。接着根据题目要求，需要筛选出 `Typrep` 字段值为“A”的数据。可以使用布尔索引（Boolean indexing）将满足条件的行筛选出来，并将结果存储在一个新的 DataFrame 中，例如 `filtered_data`。并选择表 1 中所列的特定字段。通过指定字段名的列表，使用 `[]` 运算符选择这些字段，并将结果存储在一个新的 DataFrame 中。例如，`filtered_data = filtered_data[['Accper', 'Typrep', 'Indnme', 'Prfcu', 'B001000000']]`。最后使用 `to_csv()` 函数将筛选出的数据保存为新的 CSV 文件。通过指定保存路径和文件名，以及设置 `index=False` 参数来去除保存文件时的自动索引列。

Python

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt

import seaborn as sns

import os

import warnings

warnings.filterwarnings('ignore')

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.metrics import
accuracy_score, confusion_matrix, classification_report

from sklearn.tree import DecisionTreeClassifier

import matplotlib.font_manager as fm

plt.rcParams['font.sans-serif'] = ['SimHei']

plt.rcParams['axes.unicode_minus'] = False

font = fm.FontProperties(fname='C:\Windows\Fonts\simhei.ttf', size=16)
```

运行结果如下：

```
import pandas as pd

# 读取LR.csv文件
data = pd.read_csv('LR.csv')

# 提取下列字段的数据
selected_data = data[['Stkcd', 'Accper', 'Typrep', 'B001000000', 'B001100000', 'B001101000', 'B001200000',
                    'B001201000', 'B001207000', 'B001209000', 'B001210000', 'B001211000', 'B001212000', 'B001303000', 'B002300000']]

# 筛选出字段“Typrep”值为“A”的数据
filtered_data = selected_data[selected_data['Typrep'] == 'A']

# 将筛选出的数据另存为文件“LR_1.csv”
filtered_data.to_csv('LR_1.csv', encoding='utf-8', index=False)

# 输出筛选后的数据行数、列数
num_rows, num_cols = filtered_data.shape
print(f"筛选后的数据行数: {num_rows}")
print(f"筛选后的数据列数: {num_cols}")
```

✓ 0.4s

Python

筛选后的数据行数: 33414  
筛选后的数据列数: 15

最终得到筛选后的数据行数为 33414 行，数据列数为 15 列。

### 3.2 任务 1.2

首先，我们可以使用 pandas 库提供的 read\_csv 函数分别读取 LR\_1.csv、ZCFZ.csv 和 Stk\_ind.csv 文件，并将它们存储在 DataFrame 对象中，以便后续操作。题目要求我们根据“Stkcd”、“Accper”和“Typrep”三个字段，提取“ZCFZ.csv”中字段为“A002000000”和“A001000000”的相应数据，合并到“LR\_1.csv”中。这可以通过使用 pandas 库的 merge 函数实现。同时，根据字段“Stkcd”，提取“Stk\_ind.csv”中字段为“Indnme”和“Nindnme”的相应数据，合并到“LR\_1.csv”中。这可以通过使用 pandas 库的 merge 函数实现。然后合并数据，以最终得到题目所要求的 LR\_2.csv 文件。最后保存处理后的数据。我们使用 pandas 库提供的 to\_csv 函数将得到的结果文件保存为 LR\_2.csv 文件。

Python

```
import pandas as pd
```

```
# 读取 LR_1.csv、ZCFZ.csv 和 Stk_ind.csv 文件
```



```
lr_1 = pd.read_csv('../result/LR_1.csv')

zcfz = pd.read_csv('../csv/ZCFZ.csv')

stk_ind = pd.read_csv('../csv/Stk_ind.csv', encoding='GB2312')

# 提取“ZCFZ.csv”中字段为“A002000000”和“A001000000”的相应数据
zcfz_subset = zcfz[['Stkcd', 'Accper', 'Typrep', 'A002000000',
'A001000000']]

# 合并到“LR_1.csv”中
merged_lr = pd.merge(lr_1, zcfz_subset, on=['Stkcd', 'Accper',
'Typrep'], how='left')

# 提取“Stk_ind.csv”中字段为“Indnme”和“Nindnme”的相应数据
stk_ind_subset = stk_ind[['Stkcd', 'Indnme', 'Nindnme']]

# 合并到“LR_1.csv”中
merged_lr = pd.merge(merged_lr, stk_ind_subset, on='Stkcd',
how='left')

merged_lr.head()

# 将完成合并的数据保存为 LR_2.csv
merged_lr.to_csv('../result/LR_2.csv', encoding='utf-8', index=False)

# 呈现合并后数据的行数、列数
rows, cols = merged_lr.shape
```

```
print(f"合并后的数据行数: {rows}")

print(f"合并后的数据列数: {cols}")
```

运行结果如下:

```
import pandas as pd

# 读取LR_1.csv、ZCFZ.csv和Stk_ind.csv文件
lr_1 = pd.read_csv('../result/LR_1.csv')
zcfz = pd.read_csv('../csv/ZCFZ.csv')
stk_ind = pd.read_csv('../csv/Stk_ind.csv', encoding='GB2312')

# 提取“ZCFZ.csv”中字段为“A002000000”和“A001000000”的相应数据
zcfz_subset = zcfz[['Stkcd', 'Accper', 'Typrep', 'A002000000', 'A001000000']]

# 合并到“LR_1.csv”中
merged_lr = pd.merge(lr_1, zcfz_subset, on=['Stkcd', 'Accper', 'Typrep'], how='left')

# 提取“Stk_ind.csv”中字段为“Indnme”和“Nindnme”的相应数据
stk_ind_subset = stk_ind[['Stkcd', 'Indnme', 'Nindnme']]

# 合并到“LR_1.csv”中
merged_lr = pd.merge(merged_lr, stk_ind_subset, on='Stkcd', how='left')
merged_lr.head()

# 将完成合并的数据保存为LR_2.csv
merged_lr.to_csv('../result/LR_2.csv', encoding='utf-8', index=False)

# 呈现合并后数据的行数、列数
rows, cols = merged_lr.shape
print(f"合并后的数据行数: {rows}")
print(f"合并后的数据列数: {cols}")
```

[51] ✓ 0.8s

... 合并后的数据行数: 33414  
合并后的数据列数: 19

最终得到筛选后合并的数据行数为 33414 行，数据列数为 19 列。

### 3.3 任务 1.3

首先读取 LR\_2.csv 文件。我们使用 pandas 库提供的 read\_csv 函数读取 LR\_2.csv 文件，并将它的内容储存在 DataFrame 对象 lr\_2 中。使用 `isnull().mean()` 计算每一列的空值占比，并将结果存储在变量 null\_percentage 中，其值为一个 Series 对象，索引为列名，值为对应列的空值占比。然后使用 loc 函数选取 null\_percentage 中空值占比低于 70% 的列，并存储在 filtered\_lr\_2 的 DataFrame 中。从而达到删除空值占比达 70% 及以上的列。将处理后的数据保存为 LR\_3.csv 文件。使用 to\_csv 函数将 filtered\_lr\_2 中的数据存储在 LR\_3.csv 文件中，并设置 encoding 参数为'

utf-8'，index 参数为 False 来去除自动生成的索引列。打印 filtered\_lr\_2 的信息。使用 `info()` 函数打印 filtered\_lr\_2 的相关信息，包括其列名、型号、数值类型、空值数和占比等。

```
Python

import pandas as pd

# 读取 LR_2.csv 文件

lr_2 = pd.read_csv('../result/LR_2.csv')

# 计算空值占比

null_percentage = lr_2.isnull().mean()

# 删除空值占比达 70%及以上的列

threshold = 0.7

filtered_lr_2 = lr_2.loc[:, null_percentage < threshold]

# 将处理后的数据保存为 LR_3.csv

filtered_lr_2.to_csv('../result/LR_3.csv', encoding='utf-8',
index=False)

filtered_lr_2.info()
```

运行结果如下：

```
import pandas as pd

# 读取LR_2.csv文件
lr_2 = pd.read_csv('../result/LR_2.csv')

# 计算空值占比
null_percentage = lr_2.isnull().mean()

# 删除空值占比达70%及以上的列
threshold = 0.7
filtered_lr_2 = lr_2.loc[:, null_percentage < threshold]

# 将处理后的数据保存为LR_3.csv
filtered_lr_2.to_csv('../result/LR_3.csv', encoding='utf-8', index=False)
rows, cols = filtered_lr_2.shape
print(f"合并后的数据行数: {rows}")
print(f"合并后的数据列数: {cols}")
filtered_lr_2.info()
```

[54] ✓ 0.4s

... 合并后的数据行数: 33414  
合并后的数据列数: 19  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 33414 entries, 0 to 33413  
Data columns (total 17 columns):  
# Column Non-Null Count Dtype  
---  
0 Stkcd 33414 non-null int64  
1 Accper 33414 non-null object  
2 Typrep 33414 non-null object  
3 B00100000 33414 non-null float64  
4 B00110000 33395 non-null float64  
5 B001101000 32709 non-null float64  
6 B00120000 33414 non-null float64  
7 B001201000 32686 non-null float64  
8 B001207000 33388 non-null float64  
9 B001209000 32018 non-null float64  
10 B001210000 32724 non-null float64  
11 B001211000 32732 non-null float64  
12 B001212000 32183 non-null float64  
13 A00200000 33414 non-null float64  
14 A00100000 33414 non-null float64  
15 Indnme 33414 non-null object  
16 Nindnme 33414 non-null object  
dtypes: float64(12), int64(1), object(4)

### 3.4 任务 1.4

首先读取 LR\_3.csv 文件。我们使用 pandas 库提供的 read\_csv 函数读取 LR\_3.csv 文件，并将它的内容储存在 DataFrame 对象 lr\_3 中。接着使用 dropna 函数删除包含空值的行，得到处理后的数据 lr\_4。然后将处理后的数据保存为 LR\_4.csv。使用 to\_csv 函数将 lr\_4 中的数据存储在 LR\_4.csv 文件中，并设置 encoding 参数为 'utf-8'，index 参数为 False 来去除自动生成的索引列。最后打印 lr\_4 的信息。使用 info() 函数打印 lr\_4 的相关信息，包括其列名、型号、数值类型、空值数和占比等。

```

import pandas as pd

# 读取 LR_3.csv 文件

lr_3 = pd.read_csv('../result/LR_3.csv')

# 删除包含空值的行

cleaned_lr_3 = lr_3.dropna()

# 将处理后的数据保存为 LR_4.csv

cleaned_lr_3.to_csv('../result/LR_4.csv', encoding='utf-8',
index=False)

# cleaned_lr_3.info()

```

运行结果如下：

```

import pandas as pd

# 读取LR_3.csv文件
lr_3 = pd.read_csv('../result/LR_3.csv')

# 删除包含空值的行
cleaned_lr_3 = lr_3.dropna()

# 将处理后的数据保存为LR_4.csv
cleaned_lr_3.to_csv('../result/LR_4.csv', encoding='utf-8', index=False)
rows, cols = merged_lr.shape
print(f"合并后的数据行数: {rows}")
print(f"合并后的数据列数: {cols}")
cleaned_lr_3.info()

```

[55] ✓ 0.3s

... 合并后的数据行数: 33414  
合并后的数据列数: 19  
<class 'pandas.core.frame.DataFrame'>  
Int64Index: 30888 entries, 0 to 33413  
Data columns (total 17 columns):

| #  | Column     | Non-Null Count | Dtype   |
|----|------------|----------------|---------|
| 0  | Stkcd      | 30888 non-null | int64   |
| 1  | Accper     | 30888 non-null | object  |
| 2  | Typrep     | 30888 non-null | object  |
| 3  | B001000000 | 30888 non-null | float64 |
| 4  | B001100000 | 30888 non-null | float64 |
| 5  | B001101000 | 30888 non-null | float64 |
| 6  | B001200000 | 30888 non-null | float64 |
| 7  | B001201000 | 30888 non-null | float64 |
| 8  | B001207000 | 30888 non-null | float64 |
| 9  | B001209000 | 30888 non-null | float64 |
| 10 | B001210000 | 30888 non-null | float64 |
| 11 | B001211000 | 30888 non-null | float64 |
| 12 | B001212000 | 30888 non-null | float64 |
| 13 | A002000000 | 30888 non-null | float64 |
| 14 | A001000000 | 30888 non-null | float64 |
| 15 | Indnme     | 30888 non-null | object  |
| 16 | Nindnme    | 30888 non-null | object  |

dtypes: float64(12), int64(1), object(4)  
memory usage: 4.2+ MB

### 3.5 任务 1.5

首先读取 LR\_4.csv 文件。使用 pandas 库提供的 read\_csv 函数读取 LR\_4.csv 文件，并将其内容存储在 DataFrame 对象 lr\_4 中。将字段“Accper”的日期数据转换为“YYYY-mm-dd”的格式。我们使用 pd.to\_datetime 函数将“Accper”列中的日期数据转换为 datetime 格式，然后使用 dt.strftime 函数将其格式化为“YYYY-mm-dd”的字符串格式，并将结果重新赋值给“Accper”列。将处理后的数据保存为 LR\_5.csv。使用 to\_csv 函数将 lr\_4 中的数据存储到 LR\_5.csv 文件中，并设置 encoding 参数为‘utf-8’，index 参数为 False 来去除自动生成的索引列。最后打印 lr\_4 的前几行数据。使用 head 函数打印 lr\_4 的前几行数据，默认为前 5 行。

Python

```
import pandas as pd

# 读取 LR_4.csv 文件
lr_4 = pd.read_csv('../result/LR_4.csv')

# 将字段“Accper”的日期数据转换为“YYYY-mm-dd”的格式
lr_4['Accper'] = pd.to_datetime(lr_4['Accper'], format='%Y-%m-%d').dt.strftime('%Y-%m-%d')

# 将处理后的数据保存为 LR_5.csv
lr_4.to_csv('../result/LR_5.csv', encoding='utf-8', index=False)

lr_4.head()
```

运行结果如下：

```
import pandas as pd

# 读取LR_4.csv文件
lr_4 = pd.read_csv('../result/LR_4.csv')

# 将字段"Accper"的日期数据转换为"YYYY-mm-dd"的格式
lr_4['Accper'] = pd.to_datetime(lr_4['Accper'], format='%Y-%m-%d').dt.strftime('%Y-%m-%d')

# 将处理后的数据保存为LR_5.csv
lr_4.to_csv('../result/LR_5.csv', encoding='utf-8', index=False)
lr_4.head()
```

30] ✓ 0.4s

|   | Stkcd  | Accper     | Typrep_x | B001000000   | B001100000  | B001101000  | B001200000   | B001201000   | B001207 |
|---|--------|------------|----------|--------------|-------------|-------------|--------------|--------------|---------|
| 0 | 600696 | 2018-03-31 | A        | 8.669264e+06 | 18784589.8  | 18784589.8  | 1.068992e+07 | 7.810739e+06 | 20459   |
| 1 | 547    | 2018-03-31 | A        | 9.675773e+07 | 517945246.5 | 517945246.5 | 4.266916e+08 | 2.729517e+08 | 78938   |
| 2 | 2772   | 2018-03-31 | A        | 8.953362e+07 | 267161661.1 | 267161661.1 | 1.898233e+08 | 1.527985e+08 | 69405   |
| 3 | 818    | 2018-03-31 | A        | 1.741463e+08 | 907539547.9 | 907539547.9 | 7.338691e+08 | 6.535983e+08 | 1533694 |
| 4 | 300568 | 2018-03-31 | A        | 1.069983e+08 | 171703944.2 | 171703944.2 | 1.212891e+08 | 8.066557e+07 | 287417  |

### 3.6 任务 1.6

首先，读取 LR\_5.csv 文件。使用 pandas 库提供的 read\_csv 函数读取 LR\_5.csv 文件，并将其内容存储在 DataFrame 对象 lr\_5 中；根据利润率和资产负债率的定义，使用所给公式计算出利润率和资产负债率，并将结果插入为新列，分别命名为“利润率”和“资产负债率”；使用 between 函数去除利润率和资产负债率不在  $[-300\%, 300\%]$  范围内的数据行，并将过滤后的数据集存储在 DataFrame 对象 filtered\_lr\_new 中。将处理后的数据保存为 LR\_new.csv；使用 to\_csv 函数将 filtered\_lr\_new 中的数据存储在 LR\_new.csv 文件中，并设置 encoding 参数为 'utf-8'，index 参数为 False 来去除自动生成的索引列；打印处理后的数据行数和列数。使用 shape 属性得到 filtered\_lr\_new 的行数和列数，并使用 print 函数打印；输出前 5 个企业的利润率和资产负债率。使用 head 函数得到 filtered\_lr\_new 的前 5 行数据，并输出其中的“Stkcd”、“利润率”和“资产负债率”三列。

Python

```
import pandas as pd

# 读取 LR_5.csv 文件

lr_5 = pd.read_csv('../result/LR_5.csv')

# zcfz5 = pd.read_csv('../csv/ZCFZ.csv')

# 计算利润率和资产负债率，并插入为新的列

lr_5['利润率'] = (lr_5['B001000000'] / lr_5['B001100000']) * 100

lr_5['资产负债率'] = (lr_5['A002000000'] / lr_5['A001000000']) * 100

# 删除利润率和资产负债率不在[-300%, 300%]范围内的行

filtered_lr_new = lr_5[(lr_5['利润率'].between(-300, 300)) & (lr_5['
资产负债率'].between(-300, 300))]

# 将处理后的数据保存为 LR_new.csv

filtered_lr_new.to_csv('LR_new.csv', encoding='utf-8', index=False)

# 呈现处理后的数据行数、列数

rows, cols = filtered_lr_new.shape

print(f"处理后的数据行数: {rows}")

print(f"处理后的数据列数: {cols}")

# 输出前 5 个企业的利润率和资产负债率

print("前 5 个企业的利润率和资产负债率:")

print(filtered_lr_new[['Stkcd', '利润率', '资产负债率']].head(5))
```



```
# filtered_lr_new.head()
```

运行结果如下：

```
import pandas as pd

# 读取LR_5.csv文件
lr_5 = pd.read_csv('../result/LR_5.csv')
# zcfz5 = pd.read_csv('../csv/ZCFZ.csv')

# 计算利润率和资产负债率，并插入为新的列
lr_5['利润率'] = (lr_5['B001000000'] / lr_5['B001100000']) * 100
lr_5['资产负债率'] = (lr_5['A002000000'] / lr_5['A001000000']) * 100

# 删除利润率和资产负债率不在[-300%, 300%]范围内的行
filtered_lr_new = lr_5[(lr_5['利润率'].between(-300, 300)) & (lr_5['资产负债率'].between(-300, 300))]

# 将处理后的数据保存为LR_new.csv
filtered_lr_new.to_csv('LR_new.csv', encoding='utf-8', index=False)

# 呈现处理后的数据行数、列数
rows, cols = filtered_lr_new.shape
print(f"处理后的数据行数: {rows}")
print(f"处理后的数据列数: {cols}")

# 输出前 5 个企业的利润率和资产负债率
print("前 5 个企业的利润率和资产负债率:")
print(filtered_lr_new[['Stkcd', '利润率', '资产负债率']].head(5))

# filtered_lr_new.head()
```

[60] ✓ 0.4s

```
... 处理后的数据行数: 30690
处理后的数据列数: 19
前 5 个企业的利润率和资产负债率:
   Stkcd  利润率  资产负债率
0  600696  46.150938  59.524425
1    547  18.681072  22.588596
2  2772  33.512902  34.062300
3    818  19.188844  32.569762
4  300568  62.315561  52.085464
...
```

| Stkcd  | Accper    | Typrep    | B001000000 | B001100000 | B001101000 | B001200000 | B001201000 | B001207000 | B001209000 | B001210000 |
|--------|-----------|-----------|------------|------------|------------|------------|------------|------------|------------|------------|
| 600696 | 46.150938 | 59.524425 |            |            |            |            |            |            |            |            |
| 547    | 18.681072 | 22.588596 |            |            |            |            |            |            |            |            |
| 2772   | 33.512902 | 34.062300 |            |            |            |            |            |            |            |            |
| 818    | 19.188844 | 32.569762 |            |            |            |            |            |            |            |            |
| 300568 | 62.315561 | 52.085464 |            |            |            |            |            |            |            |            |

## 四、任务二：财务数据指标分析及可视化

### 4.1 任务 2.1 根据文件绘制“行业营业利润对比分析”图

#### 4.1.1 实现思路

1. 导入 `pandas` 库：首先要导入 `pandas` 库，以便可以使用其中的函数和方法对数据进行处理和分析。

2. 读取 CSV 文件：使用 `pd.read_csv()` 函数读取目标 CSV 文件，并将其存储在一个 DataFrame 中，例如 `lr_new_data`

3. 转换日期类型：在 `lr_new_data` 中，`Accper` 列的数据可能是以字符串

的形式存储的日期数据，需要将其转换为日期类型。可以使用

`pd.to_datetime()` 函数来实现此操作，并将转换后的结果存储在 `Accper` 列中。

4. 提取年份和月份：根据需要，可以使用 `Accper` 列中的日期信息提取年份和月份，并将它们存储在新的 `Year` 和 `Month` 列中。使用 `dt.year` 和 `dt.month` 属性可以轻松地提取对应的年份和月份。

5. 过滤特定时间段的数据：根据题目要求，可能需要从数据中提取特定时间段的数据。可以使用条件筛选操作来过滤目标数据子集。例如，获取 2019 年 9 月的数据可以使用 `data_2019_sept = lr_new_data[(lr_new_data['Year'] == 2019) & (lr_new_data['Month'] == 9)]`。

6. 分组计算均值：根据要求，可能需要按照行业大类进行分组，并计算利润总额的均值。可以使用 `groupby()` 函数按照 'Indnme' 列进行分组，并使用 `mean()` 函数计算 'B001000000' 列的均值。将结果存储在一个新的 DataFrame 中。

7. 重置索引：为了方便后续处理和分析，可以使用 `reset_index()` 方法来重置结果 DataFrame 的索引。

#### 4.1.2 代码运行及结果

##### 柱状图

Python

```
import pandas as pd

# 读取 LR_new.csv 文件
lr_new_data = pd.read_csv('../result/LR_new.csv')

lr_new_data['Accper'] = pd.to_datetime(lr_new_data['Accper'])

lr_new_data['Year'] = lr_new_data['Accper'].dt.year
```

```
lr_new_data['Month'] = lr_new_data['Accper'].dt.month

# # 提取 2019 年 9 月的数据

data_2019_sept = lr_new_data[(lr_new_data['Year'] == 2019) &
(lr_new_data['Month'] == 9)]

# # 按行业大类进行分组，并计算利润总额的均值

mean_profit_by_industry =
data_2019_sept.groupby('Indnme')['B001000000'].mean().reset_index()
```

```
Python

import pandas as pd

import matplotlib.pyplot as plt

# 读取包含利润总额和行业大类的数据集

profits_data = pd.read_csv('LR_new.csv', encoding='utf-8')

# 按行业大类分组，计算利润总额的均值

mean_profits_by_industry =
profits_data.groupby('Indnme')['B001000000'].mean()

# 绘制柱状图

mean_profits_by_industry.plot(kind='bar', figsize=(10, 6),
color='skyblue')

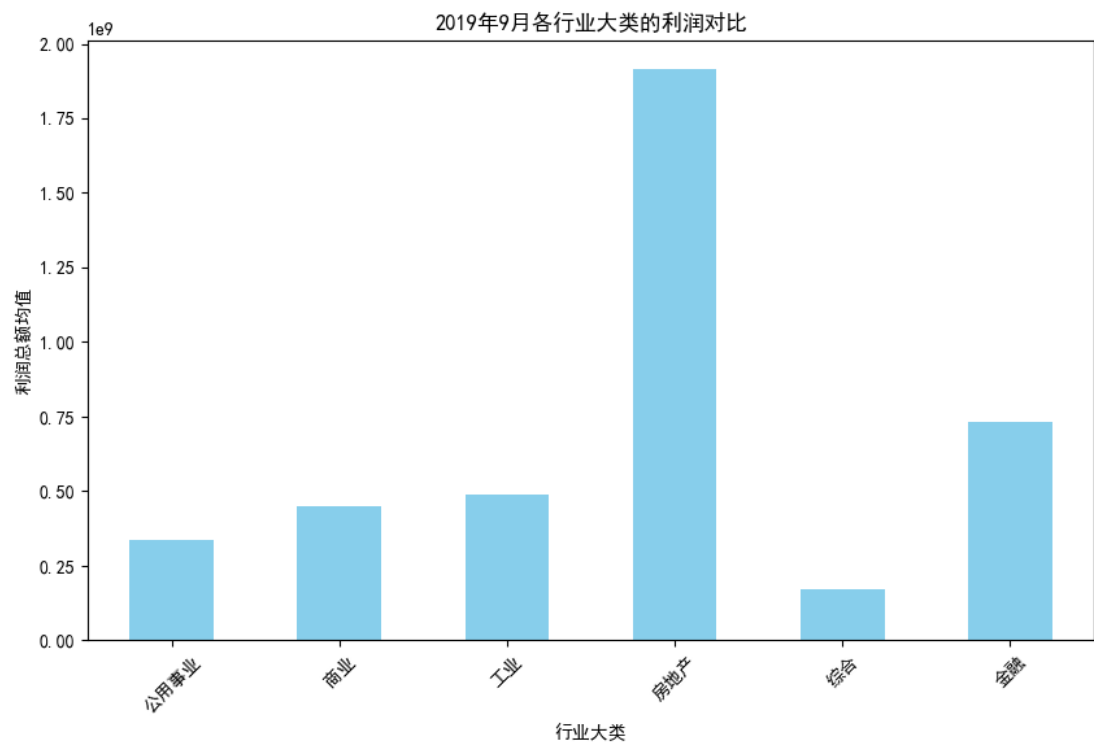
plt.title('2019 年 9 月各行业大类的利润对比')
```

```
plt.xlabel('行业大类')

plt.ylabel('利润总额均值')

plt.xticks(rotation=45)

plt.show()
```



### 折线图:

```
Python

import pandas as pd

# 读取 LR_new.csv 文件
lr_new_data = pd.read_csv('../result/LR_new.csv')

lr_new_data['Accper'] = pd.to_datetime(lr_new_data['Accper'])

lr_new_data['Year'] = lr_new_data['Accper'].dt.year
```

```
lr_new_data['Month'] = lr_new_data['Accper'].dt.month

# # 提取 2019 年 9 月的数据

data_2019_sept = lr_new_data[(lr_new_data['Year'] == 2019) &
(lr_new_data['Month'] == 9)]

# # 按行业大类进行分组，并计算利润总额的均值

mean_profit_by_industry =
data_2019_sept.groupby('Indnme')['B001000000'].mean().reset_index()
```

Python

```
import pandas as pd

import matplotlib.pyplot as plt

# 读取数据

lr_new_data = pd.read_csv('LR_new.csv')

# 将日期列转换为日期时间类型

lr_new_data['Accper'] = pd.to_datetime(lr_new_data['Accper'])

# 提取年份和季度信息

lr_new_data['Year'] = lr_new_data['Accper'].dt.year

lr_new_data['Quarter'] = lr_new_data['Accper'].dt.quarter
```

```
# 根据表达式计算利润率

lr_new_data['Profit_Rate'] = lr_new_data['B001000000'] /
lr_new_data['B001100000']


# 按行业大类、年份和季度进行分组，并计算利润率均值

mean_profit_by_industry = lr_new_data.groupby(['Indnme', 'Year',
'Quarter'])['Profit_Rate'].mean().reset_index()


# 绘制折线图

plt.figure(figsize=(12, 8))


# 循环绘制每个行业大类的利润率均值变化折线图

for industry in mean_profit_by_industry['Indnme'].unique():

    temp_data =
mean_profit_by_industry[mean_profit_by_industry['Indnme'] == industry]

    plt.plot(temp_data['Year'] + temp_data['Quarter']/10,
temp_data['Profit_Rate'], marker='o', label=industry)

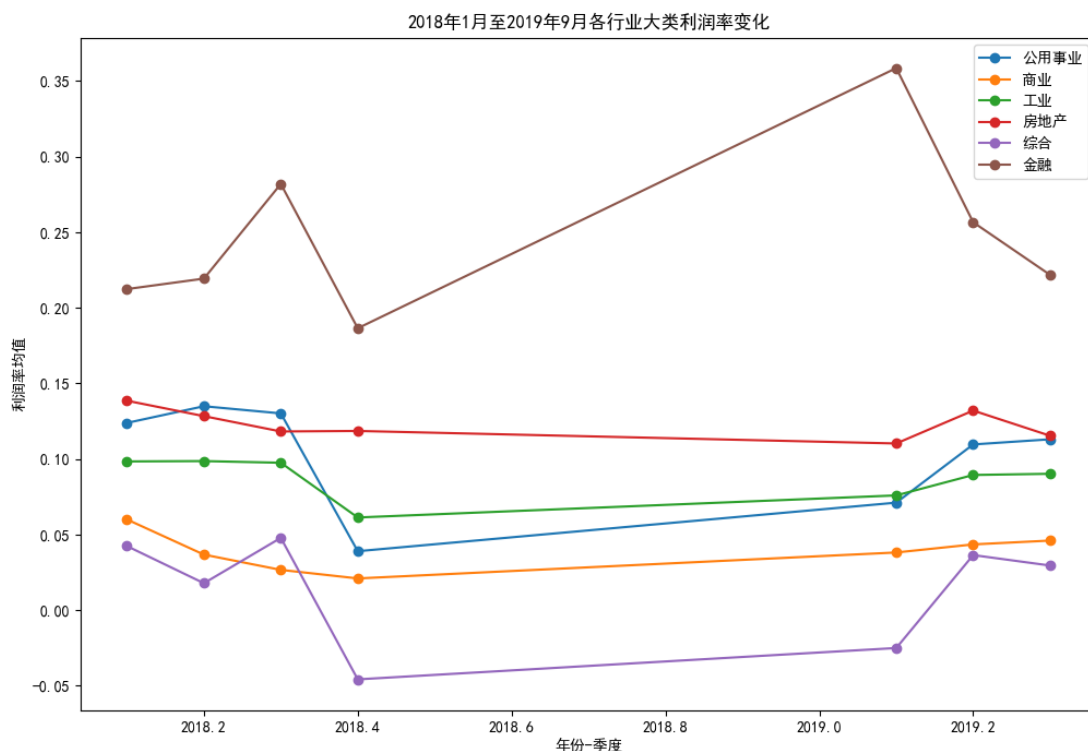

plt.title('2018 年 1 月至 2019 年 9 月各行业大类利润率变化')

plt.xlabel('年份-季度')

plt.ylabel('利润率均值')

plt.legend()

plt.show()
```



### 4.1.3 结果分析

**柱状图：**展示了 2019 年 9 月各行业大类的利润总额均值的对比情况。下面是对该图表的分析：

- 行业大类利润对比：**柱状图清晰地展示了不同行业大类在 2019 年 9 月的利润均值对比情况。每个柱状条代表一个行业大类，其高度表示该行业在该月份的利润总额均值，这种形式直观地呈现了各行业之间的盈利差异。
- 行业利润差异明显：**从柱状图可以看出，各行业之间的利润均值存在明显的差异。有些行业的柱状条较高，表示其利润均值较高，而其他行业的柱状条则相对较低，这反映了不同行业在盈利能力上的差异。
- 行业利润趋势：**通过比较各行业的柱状条，还可以观察到不同行业在利润均值上的趋势。一些行业的柱状条可能呈现增长趋势，而其他行业则可能呈现下降趋势，这为分析行业发展趋势提供了重要线索。
- 数据可视化效果：**柱状图通过直观的方式展示了行业利润的对比情况，使数据更易于理解和比较。同时，图表的标题、坐标轴标签和颜色选择也使得信息传达更加清晰。

基于以上分析，这份柱状图为分析 2019 年 9 月各行业大类的利润情况提供了重要参考。投资者、政策制定者和企业经营者可以通过这份图表更好地了解行业利润的对比情况，制定相应的投资策略、经济政策和经营决策，以促进经济发展和行业竞争力的提升。同时，结合其他统计数据和分析方法，将有助于更深入地理解行业利润的变化规律，为相关决策提供更全面的依据。

**折线图：**展示了 2018 年 1 月至 2019 年 9 月期间各行业大类利润率均值的变化情况。图表通过折线图的形式清晰地展现了不同行业的盈利状况，为企业和机构提供了重要的经济分析信息。

首先，图表的横轴表示时间，以年份-季度的形式呈现，纵轴表示利润率均值。每条折线代表一个行业大类，通过折线的波动可以直观地观察到不同行业在利润率方面的变化趋势。

从图表中可以观察到以下几个重要的趋势和现象：

1. 行业之间的差异：不同行业大类的利润率均值存在明显的差异。有些行业的利润率整体较高，而另一些行业的利润率则相对较低，这反映了不同行业在盈利能力上的差异。

2. 季节性变化：部分行业的利润率在不同季度之间出现周期性的波动。这种季节性变化可能受到季节性消费、生产周期等因素的影响，为企业制定季节性经营策略提供了重要依据。

3. 总体趋势：除了季节性波动外，各行业大类的利润率均值还呈现出总体的趋势。有些行业的利润率呈现稳定增长或下降的趋势，而另一些行业则波动较大，这可能与宏观经济政策、市场竞争等因素有关。

4. 行业间比较：通过折线图的比较，可以清晰地观察到不同行业大类利润率均值的相对优势和劣势，为投资者、政策制定者以及企业经营者提供了重要的参考依据。

综合而言，该图表为分析 2018 年 1 月至 2019 年 9 月期间各行业大类的利润率提供了直观且全面的信息。基于这些信息，相关方可以更好地了解不同行业的盈利状况，制定相应的经济政策、投资决策和企业经营策略，促进经济发展和企业竞争力的提升。



## 4.2 任务 2.2 根据文件绘制该行业大类相关的“行业企业营收分析”图

### 4.2.1 实现思路

使用了 Pandas 和 Matplotlib 库对 CSV 文件中的数据进行处理和可视化，主要包括以下几个步骤：

1. 读取 CSV 文件：使用 `pd.read_csv` 函数读取名为 'LR\_new.csv' 的 CSV 文件中的数据，并将其存储在名为 `data` 的 DataFrame 中。

2. 计算营业利润率：通过对 `data` 中的营业收入和营业利润进行计算，得到了每个样本的营业利润率，并将其添加到 `data` 中的新列 '利润率' 中。

3. 按行业大类分组计算均值：通过 `groupby` 函数，根据行业大类 ('Indnme') 对数据进行分组，然后计算每个行业大类的利润率均值，并按降序进行排序，将结果存储在 `mean_profit_rate` 中。

4. 确定排名第一的行业大类：从 `mean_profit_rate` 中获取利润率均值最高的行业大类。

5. 筛选出排名第一的行业大类数据：从原始数据 `data` 中筛选出属于排名第一行业大类的数据，存储在 `top_category_data` 中。

6. 计算不同细类的利润率：针对排名第一的行业大类数据，再次按照行业细类 ('Nindnme') 进行分组，计算每个行业细类的利润率均值并按降序排序，结果存储在 `mean_subcategory_profit_rate` 中。

7. 绘制利润率排名前三细类的柱状图：从 `mean_subcategory_profit_rate` 中选取前三个行业细类，利用 Matplotlib 库绘制柱状图，展示了这些行业细类的利润率情况，并添加了相应的标题、坐标轴标签。最后通过 `plt.show()` 显示出图表。

8. 对企业“T1”在 2019 年 3 月、6 月、9 月的数据进行了筛选，并计算了利润率和资产负债率。其中，利润率的计算公式为营业总收入除以营业总成本再乘以 100，资产负债率的计算公式为负债总额除以资产总额再乘以 100。利用

Matplotlib 库绘制了营业总成本占比的饼图，并计算了营业总成本、营业总收入、利润率和资产负债率的相关数值并进行了打印输出。最后再结合 Excel 表格生成 2019 年企业“T1”经营情况分析柱状折线组合图。

#### 4.2.2 代码运行及结果

确定排名第一的行业大类

```
Python

import pandas as pd

import matplotlib.pyplot as plt

# 读取 CSV 文件

data = pd.read_csv('../result/LR_new.csv')

# 计算营业利润率

data['利润率'] = (data['B001000000'] / data['B001100000']) * 100

# 按行业大类分组计算均值

mean_profit_rate = data.groupby('Indnme')['利润率']
                        .mean().sort_values(ascending=False)

# 确定排名第一的行业大类

top_category = mean_profit_rate.index[0]

# 筛选出排名第一的行业大类数据

top_category_data = data[data['Indnme'] == top_category]
```

```
top_category_data.head()
```

Python

```
import pandas as pd

import matplotlib.pyplot as plt

# 读取 CSV 文件

data = pd.read_csv('../result/LR_new.csv')

# 计算营业利润率

data['利润率'] = (data['B001000000'] / data['B001100000']) * 100

# 按行业大类分组计算均值

mean_profit_rate = data.groupby('Indnme')['利润率'
'].mean().sort_values(ascending=False)

# 确定排名第一的行业大类

top_category = mean_profit_rate.index[0]

# 筛选出排名第一的行业大类数据

top_category_data = data[data['Indnme'] == top_category]

top_category_data.head()
```

```
# 计算不同细类的利润率

mean_subcategory_profit_rate = top_category_data.groupby('Nindnme')['
利润率'].mean().sort_values(ascending=False)

# 绘制利润率排名前三细类的柱状图

top3_mean_profit_rate = mean_subcategory_profit_rate.head(3)

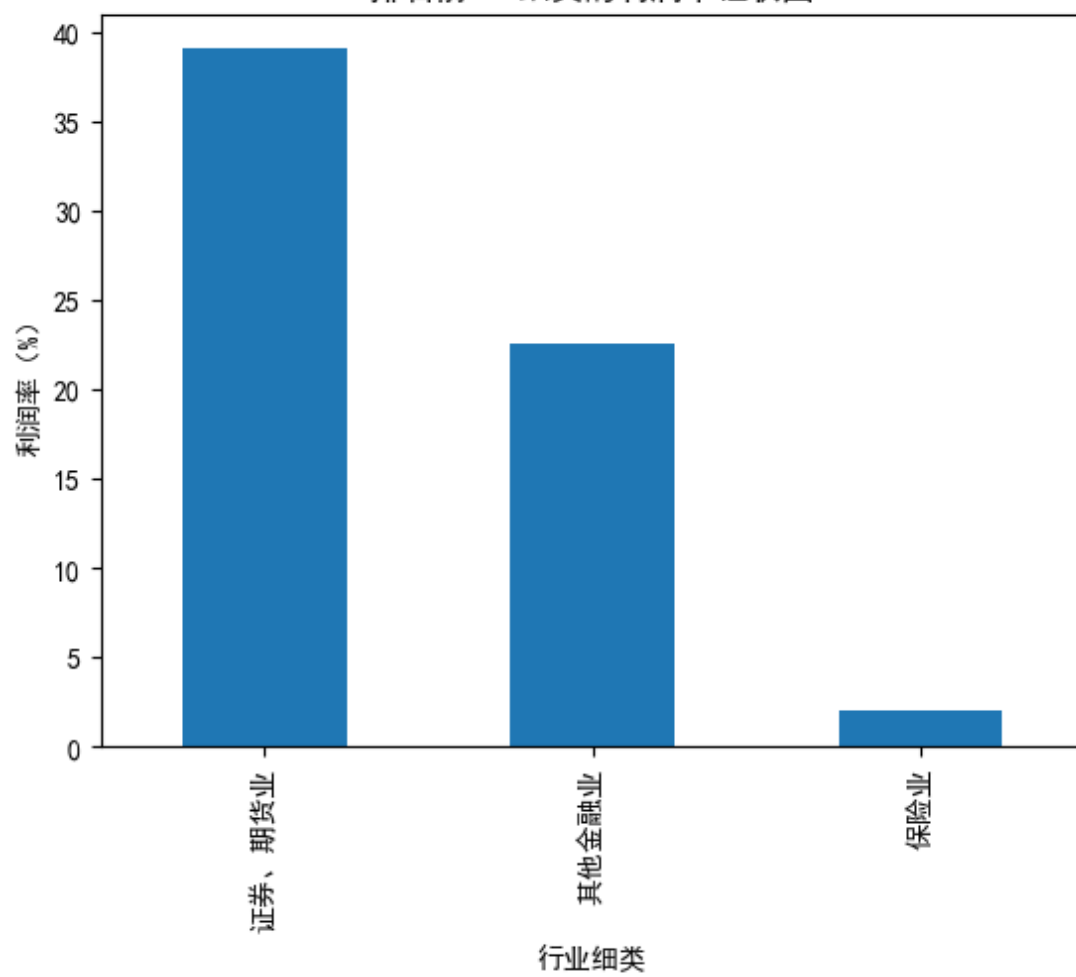
top3_mean_profit_rate.plot(kind='bar', title='排名前 3 细类的利润率柱
状图')

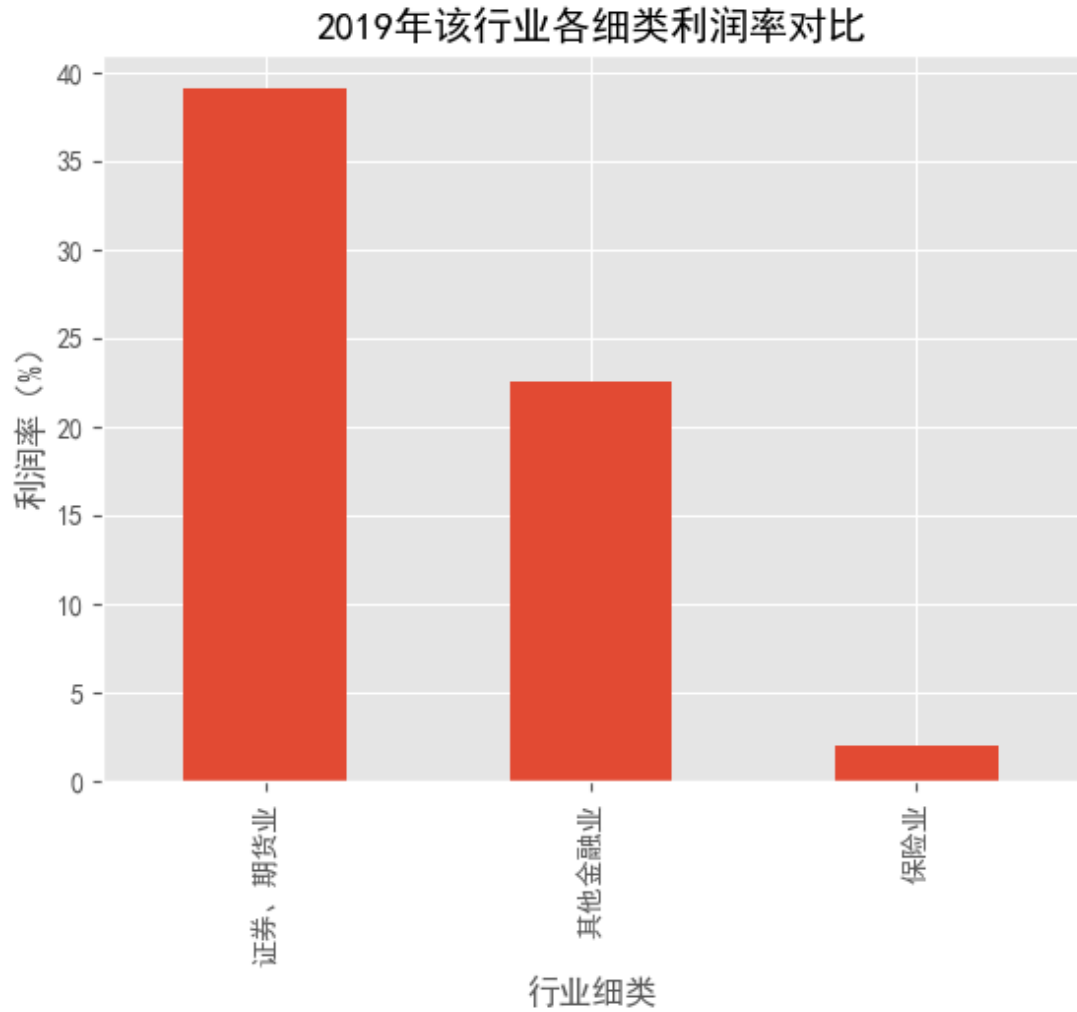
plt.xlabel('行业细类')

plt.ylabel('利润率 (%)')

plt.show()
```

排名前 3 细类的利润率柱状图





Python

```
import pandas as pd

import matplotlib.pyplot as plt

# 读取 CSV 文件

data = pd.read_csv('../result/LR_new.csv')

data['Accper'] = pd.to_datetime(data['Accper'])

data['Year'] = data['Accper'].dt.year
```

```
data['Month'] = data['Accper'].dt.month

data_2019_sept = data[(data['Year'] == 2019) & (data['Month'] == 9)]

# 计算营业利润率

data_2019_sept['利润率'] = (data_2019_sept['B001000000'] /
data_2019_sept['B001100000']) * 100


# # 按行业大类分组计算均值

mean_profit_rate = data_2019_sept.groupby('Indnme')['利润率'
'].mean().sort_values(ascending=False)


# # 确定排名第一的行业大类

top_category = mean_profit_rate.index[0]


# # 筛选出排名第一的行业大类数据

top_category_data = data_2019_sept[data_2019_sept['Indnme'] ==
top_category]


# # 筛选出该行业大类中利润率排名第 1 的细类数据

top_subcategory_data = data_2019_sept[(data_2019_sept['Indnme'] ==
top_category) & (data_2019_sept['Nindnme'] == '证券、期货业')]


# # 对该细类企业数据按利润率进行排序

top_subcategory_data_sorted = top_subcategory_data.sort_values('利润率',
ascending=False)
```

```
# # 取前 5 家企业的利润率数据

top5_profit_rate = top_subcategory_data_sorted.head(5)

lst1=['T1','T2','T3','T4','T5']

lst2=list(top5_profit_rate['Stkcd'])

for i in range(5):

    top5_profit_rate['Stkcd'].replace({lst2[i]:lst1[i]}, inplace=True)


# 绘制利润率排名前五的企业的柱状图

plt.bar(top5_profit_rate['Stkcd'], top5_profit_rate['利润率'], color='skyblue')

plt.title('2019 年该行业利润率排名第 1 细类的企业利润率对比')

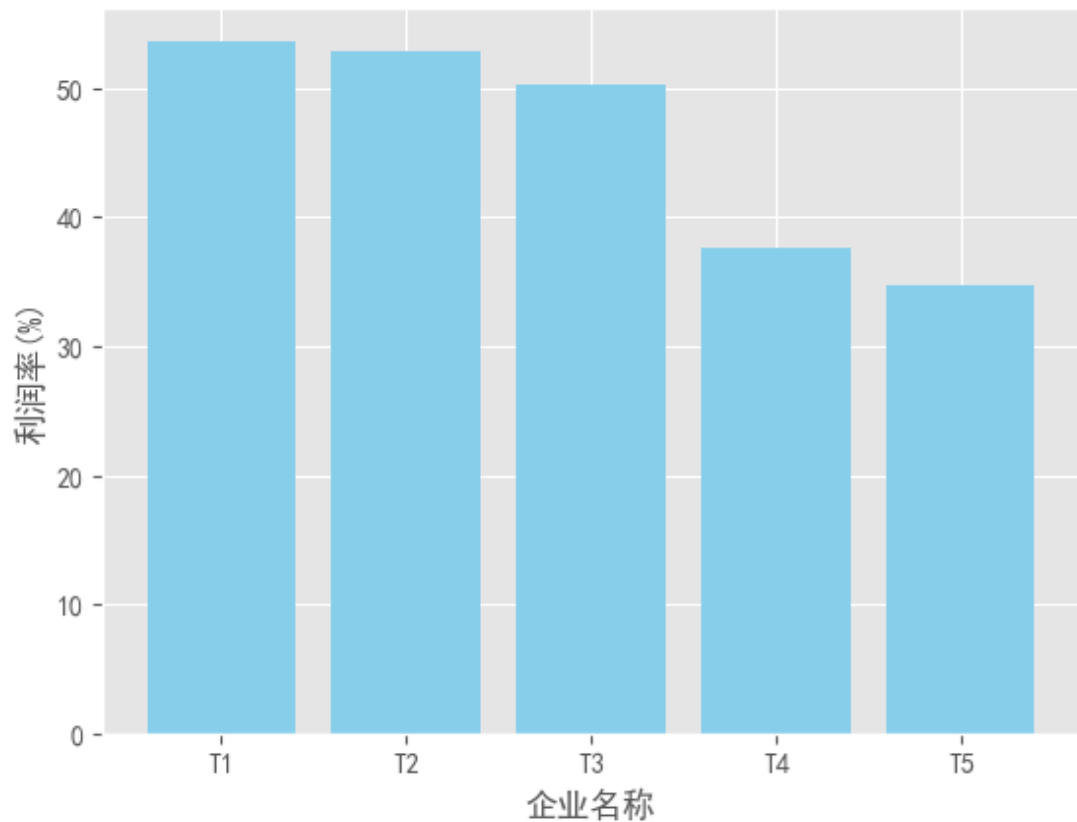
plt.xlabel('企业名称')

plt.ylabel('利润率(%)')

plt.show()
```



2019 年该行业利润率排名第1细类的企业利润率对比



Python

#得到要求的数据

```
fina_items=['B001209000','B001210000','B001211000','B001201000','B001207000']
```

```
labels=['销售费用','管理费用','财务费用','营业成本','营业税金及附加']
```

```
t1_2019=top5_profit_rate[top5_profit_rate['Stkcd']=="T1"]
```

#选出营业成本、营业税金及附加、销售费用、管理费用、财务费用

```
select_data=t1_2019[fina_items]
```

```
value1=1.669572e+09 #营业总成本
```

#费用转为一维列表，嵌入饼图中

```
percentages=(select_data/value1*100).values.flatten()

colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue',
'lightgreen']

# 绘制饼图

plt.figure(figsize=(10,8))

plt.pie(percentages, labels=labels, autopct='%.1f%%', colors=colors,
shadow=True)

plt.legend(loc="upper left")

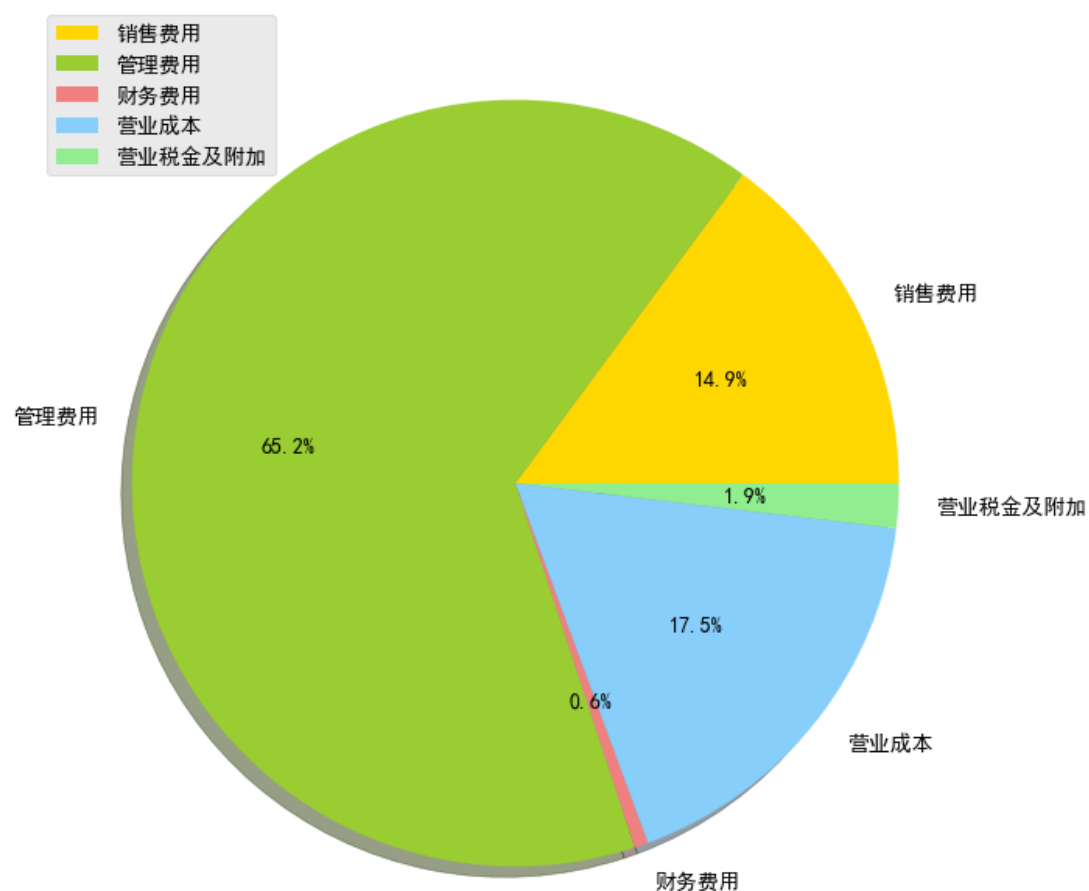
# 添加标题

plt.title('2019 年企业 “T1” 营业总成本分析')

# 显示饼图

plt.show()
```

2019年企业“T1”营业总成本分析



Python

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.dates as mdates
```

```
# 读取 CSV 文件
```

```
data = pd.read_csv('../result/LR_new.csv')
```

```
data.loc[data['Stkcd'] == 300059, 'Stkcd'] = 'T1'
```

```
# print(data.loc[data['Stkcd'] == 'T1', 'Stkcd'])
```

```
data['Accper'] = pd.to_datetime(data['Accper'])

# data.info()

# # 筛选出企业“T1”在2019年3月、6月、9月的数据

t1_2019_quarters = data[(data['Stkcd'] == 'T1') &
(data['Accper'].dt.month.isin([3, 6, 9])) & (data['Accper'].dt.year ==
2019)]

# t1_2019_quarters.head()


# # 计算利润率

t1_2019_quarters['利润率'] = (t1_2019_quarters['B001000000'] /
t1_2019_quarters['B001100000']) * 100


# # 计算资产负债率

t1_2019_quarters['资产负债率'] = (t1_2019_quarters['A002000000'] /
t1_2019_quarters['A001000000']) * 100


# print(t1_2019_quarters['B001100000'])

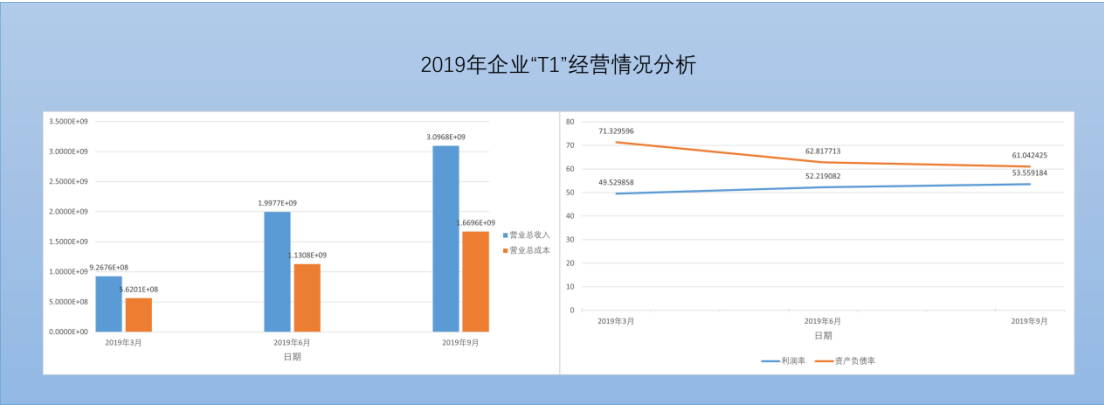
# # 绘制营业总收入和营业总成本的柱状图


print(t1_2019_quarters['B001200000']) # 营业总成本

print(t1_2019_quarters['B001100000']) # 营业总收入


print(t1_2019_quarters['利润率'])

print(t1_2019_quarters['资产负债率'])
```



4. 2. 3 结果分析

2019 年该行业各细 类利润率对比图

1. 排名前 3 细类的利润率对比：

柱状图清晰地展示了排名前三细类的利润率情况，有助于对比不同细类的盈利能力。

2. 行业细类利润率差异：

通过柱状图可以观察到排名前三细类的利润率差异，有助于发现行业内部不同细类的盈利情况，为进一步分析提供了依据。

3. 决策参考：

该柱状图可以为企业管理者提供有关行业内不同细类的利润率对比，有助于评估企业在不同细分市场的竞争地位，指导企业制定相应的战略决策。

综上所述，以上柱状图为企业管理者提供了有关排名前三细类的利润率对比，有助于深入了解行业内部不同细类的盈利能力，为制定战略决策提供了重要参考依据。

2019 年该行业利润率排名第 1 细类的企业利润率对比图

1. 行业利润率排名：

通过柱状图可以清晰地看出 2019 年在特定行业中，证券、期货业的利润率排名第一。这表明证券、期货业在该行业中取得了较高的利润率，具有较好的盈利能力。

## 2. 企业利润率对比：

柱状图对比了该行业利润率排名前五的企业，从中可以看出各家企业的利润率情况。这有助于分析各家企业在行业内的竞争优势和盈利能力。

## 3. 企业标识：

为了保护企业隐私，对企业名称进行了编码处理，分别用 T1、T2、T3、T4 和 T5 代表排名前五的企业。

## 4. 决策参考：

该柱状图可以为企业管理者提供有关行业内各企业利润率的直观对比，有助于评估企业在行业内的竞争地位，并可以作为制定战略决策的参考依据。

综合来看，以上柱状图对于分析特定行业中利润率排名前五的企业提供了重要的数据展示和比较，有助于企业管理者深入了解行业竞争格局，指导企业制定相应的经营战略。

## 2019 年企业“T1” 营业总成本分析图

### 1. 构成比例：

通过饼图可以看出，2019 年企业“T1” 营业总成本中，营业成本所占比例较大，其次是销售费用和管理费用，而财务费用和营业税金及附加的比例相对较小。

### 2. 主要支出项目：

营业成本在整个成本结构中所占比例较大，这可能意味着企业在生产和销售过程中所发生的直接成本较高，需要重点关注成本管控和效率提升。

销售费用和管理费用也占据一定比例，说明企业在销售和管理方面的支出比较显著，需要关注这两个方面的成本管理和效益提升。

### 3. 财务稳健性：

相对而言，财务费用和营业税金及附加所占比例较小，可能表明企业在财务运营和税务方面相对稳健，但仍需密切监控以确保财务稳健性。

### 4. 决策参考：

该饼图为企业管理者提供了直观的财务数据分析，有助于他们更好地理解企业财务结构，找出成本结构中的关键项目，并针对性地采取措施来优化成本管理和提升效益。

综合来看，以上饼图分析为企业管理层提供了关于成本结构的重要信息，有助于他们制定相应的管理决策，优化资源配置，提高经营效率及盈利能力。

## 2019 年企业“T1” 经营情况分析图

### 1. 营业总成本和营业总收入柱状图：

打印了企业“T1”在2019年3月、6月、9月的营业总成本和营业总收入数据，并打算进行柱状图的绘制。这两个数据集可以用于分析企业的经营成本和盈利情况，从而帮助评估企业的盈利能力和经营效率。柱状图可以直观地展示出不同季度的营业总成本和营业总收入的变化情况，进而帮助发现潜在的经营问题或优势。

### 2. 利润率和资产负债率的数据输出：

打印了企业“T1”在2019年3月、6月、9月的利润率和资产负债率数据。利润率是企业盈利能力的重要指标，资产负债率则可以反映企业的偿债能力和财务风险。通过对这些数据的分析，可以更全面地了解企业的财务状况和经营状况，帮助投资者和管理者做出更明智的决策。

基于数据处理操作，接下来利用Excel绘制利润率和资产负债率随时间变化的折线图，以便更清晰地观察这些指标的变化趋势。

## 4.3 任务 2.3 绘制“行业与企业营业数据分析”的可视化大屏

### 4.3.1 实现思路

确定数据源，通过任务二的前半部分得到了所有的数据图表，接下来通过html语言来绘制大屏展示，将图表使用2\*3的排列方式进行排列，生成大屏图像。

### 4.3.2 代码运行及结果

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
body{
```

```
    margin: 0;
```

```
    padding: 0;
```

```
    overflow: hidden;
```

```
    background: linear-gradient(to bottom right, #1e5799,  
#7db9e8);
```

```
}
```

```
.title{
```

```
    text-align: center;
```

```
    font-size: 36px;
```

```
    margin-bottom: 20px;
```

```
    color: #ffffff; /* 设置标题颜色为白色 */
```

```
    text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5); /* 添加文字  
阴影效果 */
```

```
}
```

```
.container {
```

```
    display: grid;
```

```
    grid-template-columns: repeat(3, 1fr);
```

```
    grid-gap: 20px;
```

```
    justify-items: center;
```



```
        align-items: center;

        height: 100vh; /* 设置容器高度为视口高度，填满整个屏幕 */
    }

    .image {

        max-width: 100%; /* 图像宽度不超过容器宽度 */

        max-height: 100%; /* 图像高度不超过容器高度 */

    }

</style>

</head>

<body>

    <div class="title">

        行业与企业营业数据分析

    </div>

    <div class="container">

         <!-- 替换为实际图片的路径 -->

    </div>

</body>

</html>
```

</div>

</body>

</html>



### 4.3.3 结果分析

在可视化大屏上展示分析结果的同时，结合文字说明和数据标注，对于突出的数据点或者变化趋势进行解读和分析，帮助观众快速理解数据背后的含义。

如果条件允许，可以添加交互功能，比如通过点击或者下拉菜单切换不同的行业数据，或者查看特定企业的详细数据信息，提升数据展示的交互性和灵活性。

如果是基于实时数据的大屏展示，需要考虑数据的实时更新和监控机制，确保数据的及时性和准确性。

可以通过自动化脚本定时更新数据，并设置报警机制监控数据异常，保证大屏数据的稳定展示。

通过以上步骤，可以实现“行业与企业营业数据分析”的可视化大屏设计与展示，帮助用户全面了解企业与行业的营业情况，从而做出更加科学合理的

决策。

## 五、任务三：企业利润预测及财务造假识别

### 5.1 任务 3.1

#### 5.1.1 实现思路

首先，通过 `pd.read_csv('financial_data.csv')` 从 'financial\_data.csv' 文件中读取数据到 DataFrame 变量 `df` 中。

接下来，使用 `df.corr()` 计算了整个数据集中各列的相关性系数，生成了一个相关性矩阵 `corr`。

然后，通过 `corr_sort = corr.sort_values(by="LRZE", ascending=False)` 对相关性矩阵进行排序，按照“LRZE”列的相关性系数降序排列，得到了排序后的相关性矩阵 `corr_sort`。

最后，通过 `for i in corr_sort['LRZE'][1:6].index:` 遍历输出了与“LRZE”列具有最强正相关性的前 5 个特征的索引值。

根据得出的 5 个索引值，结合 Excel 表格得出相关度最高的 5 个指标。

#### 5.1.2 实现过程

```
> ~
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import warnings
warnings.filterwarnings('ignore')
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.tree import DecisionTreeClassifier
import matplotlib.font_manager as fm

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

font = fm.FontProperties(fname='C:\\Windows\\Fonts\\simhei.ttf', size=16)

[1]

> ~
df = pd.read_csv('../csv/financial_data.csv')
corr=df.corr()
type(corr)
corr_sort = corr.sort_values(by="LRZE",ascending=False)
for i in corr_sort['LRZE'][1:6].index:
    print(i)

[7] ✓ 0.0s

..
..
YYSR
YWFY
YYCB
YYSJJFJ
ZCJZSS
```

Python

```
df = pd.read_csv('../csv/financial_data.csv')

corr=df.corr()

type(corr)

corr_sort = corr.sort_values(by="LRZE", ascending=False)

for i in corr_sort['LRZE'][1:6].index:

    print(i)
```

### 5.1.3 计算结果

| 相关系数     | YYSR     | 相关系数     | YYCB     | 相关系数     | YYSJJFJ  | 相关系数    | YWFY     | 相关系数     | ZCJZSS   | 相关系数     |
|----------|----------|----------|----------|----------|----------|---------|----------|----------|----------|----------|
| -0.08437 | 4.8E+08  | 0.782726 | 2.94E+08 | 0.737736 | 1505596  | 0.56544 | 70399459 | 0.772832 | 23119755 | 0.238524 |
|          | 5.33E+09 |          | 3.5E+09  |          | 28213514 |         | 1.37E+09 |          | 29930865 |          |
|          | 1.68E+09 |          | 7.88E+08 |          | 8298888  |         | 5.86E+08 |          | 1766300  |          |
|          | 2.23E+09 |          | 1.74E+09 |          | 11585500 |         | 3.94E+08 |          | 4007796  |          |
|          | 3.95E+08 |          | 2.99E+08 |          | 1621818  |         | 37244011 |          | 970097   |          |
|          | 5.23E+08 |          | 4.38E+08 |          | 3748919  |         | 66226872 |          | 636576.2 |          |
|          | 1.66E+08 |          | 97225559 |          | 1058506  |         | 28423480 |          | 2511442  |          |
|          | 4.48E+08 |          | 3.07E+08 |          | 2930419  |         | 51948515 |          | 730273.5 |          |
|          | 9.76E+08 |          | 6.55E+08 |          | 7485361  |         | 97431050 |          | 45979463 |          |
|          | 4.55E+09 |          | 3.65E+09 |          | 20112934 |         | 5.02E+08 |          | 5431486  |          |

根据最终计算结果可以选取相关度最高的 5 个指标，分别是 YYSR（营业收入）、YWFY（业务费用）、YYCB（营业成本）、YYSJJFJ（营业税金及附加）、ZCJZSS（资产减值损失）

## 5.2 任务 3.2

### 5.2.1 预测模型构建

企业利润的预测，是在销售预测的基础上，通过对产品产销量、价格、结构等因素进行综合分析计算，测算企业未来期间可能达到的利润水平及其变动趋势。它是会计预测的重要内容并且其预测方法的选择是否恰当直接关系到预测的准确性和财务预算的实用性。针对此问题，我们选择采用 BP（Back - Propagation）神经网络建立预测模型，这种网络有着其显著的优越性，即在建模预测过程中，只需要知道历史数据，通过这些历史数据构造出一个模型，然后进行预测，这样可以避免需要知道状态方程的麻烦，也可以避免陷入局部极小。

BP 网络的基本思想。BP 神经网络是误差反向传播多层前向反馈式神经网络，它是按层次结构构造的，包括一个输入层、一个输出层和一个或多个隐含层，一层内的节点（神经元）只和与该层紧邻的下一层的各节点连接。这个网络的学习过程由正向传播和反向传播两个过程组成。在正向传播过程中，输入信息从输入层经隐含层逐层处理，并向后传向输出层，每一层神经元的状态只影响下一层神经元的状态。如果在输出层不能得到期望的输出，则转向反向传播过程，将误差信号沿原来的连接通路返回，通过修改各层神经元的权值，使得误差信号最小。实际上，BP 模型把一组样本的输入输出问题变为一个非线性优化问题。我们可以把这种模型看成一个从输入到输出的映射，这个映射是高

度非线性的。如果输入节点数为  $n$ ，输出节点数为  $m$ ，则神经网络表示的是从  $n$  维欧氏空间到  $m$  维欧氏空间的映射。

利润预测的 BP 算法。利用 BP 算法进行利润预测与常规方法（如回归分析）不同，常规方法是根据提供的数据找出描述利润与影响其变化的诸多因素之间关系的函数表达式，然后进行预测；而 BP 算法则是把利润与影响其变化的诸多因素之间的映射关系隐含在网络结构及其权值上，经过学习确定具有较好预测功能的网络结构及权值，此时的神经网络等同于确定的函数表达式，可用来进行利润预测。

### 5.2.2 实现过程

建立基于 BP 神经网络的企业利润预测模型需要经过数据准备、神经网络设计、模型训练和评估等多个步骤。以下是一个详细的解释：

#### 1. 数据准备

首先，收集历史数据，包括 YYSR（营业收入）、YWFY（业务费用）、YYCB（营业成本）、YYSJJFJ（营业税金及附加）、ZCJZSS（资产减值损失）等指标作为输入变量，同时收集相应的利润数据作为输出变量。对数据进行清洗、归一化处理，将数据集分为训练集和测试集。

#### 2. 神经网络设计

**输入层：**神经网络的输入层节点数为 5，分别对应 YYSR、YWFY、YYCB、YYSJJFJ、ZCJZSS 这五个指标。

**隐藏层：**根据实际情况和试验结果，选择合适数量的隐藏层和每个隐藏层的神经元数量。可以通过交叉验证等方法确定最佳的网络结构。

**输出层：**输出层节点为 1，表示预测的企业利润。

#### 3. 模型训练

**初始化权重和偏置：**对神经网络的权重和偏置进行初始化。

**前向传播：**采用前向传播算法计算神经网络的输出。

**反向传播：**根据预测输出和实际输出之间的误差，利用反向传播算法调整

权重和偏置，以减小预测误差。

**优化算法：**选择合适的优化算法，如梯度下降算法，并设置学习率等超参数。

**停止条件：**设置停止条件，如达到最大迭代次数或误差小于某个阈值时停止训练。

4. 模型评估

**测试集验证：**使用测试集数据验证模型的泛化能力，评估模型在未见过的数据上的预测表现。

**性能指标：**计算预测误差指标，如均方误差（MSE）、平均绝对误差（MAE）等，评估模型的预测准确性。

5. 结果解释和展示

解释神经网络模型对企业利润的影响因素，呈现模型预测结果，并提供决策支持。

6. 模型优化和更新

根据实际情况不断优化模型，更新数据，以适应企业经营环境的变化。

在建立神经网络模型的过程中，需要认真选择网络结构、合适的激活函数、优化算法等，并对模型进行充分的训练和验证，以确保其具有良好的预测能力和稳健性。

5.2.3 预测结果

|    | D        | E        | F        | G           | H          | I         | J          | K          | L          | M                             | N                         |
|----|----------|----------|----------|-------------|------------|-----------|------------|------------|------------|-------------------------------|---------------------------|
| 1  | YVIFY    | ZCJZSS   | LRZE     | 五个公司各指标的平均数 |            |           |            |            |            |                               | 0.92838964 五个指标系数（五元一次方程） |
| 2  | 70399459 | 23119755 | 1.25E+08 | 4840004339  | 3965819098 | 452499498 | 5847836705 | 4019928501 | 2988756549 | -0.917938628                  | 可以粗略计算test.csv中的利润        |
| 3  | 1.37E+09 | 29930865 | 4.15E+08 | 4606653227  | 3720792018 | 420680265 | 6065519949 | 4063323273 | 2911254374 | -1.227352587                  |                           |
| 4  | 5.86E+08 | 1766300  | 3.84E+08 | 4668777489  | 3761966973 | 47042296  | 6467698775 | 5318693814 | 2832652408 | -0.941539178                  |                           |
| 5  | 3.94E+08 | 4007796  | 1.22E+08 | 4999388692  | 3941063875 | 575069914 | 6935442221 | 540963885  | 3700934851 | 1.293025229                   |                           |
| 6  | 37244011 | 970097   | 60426742 | 5668524868  | 4434877027 | 629913751 | 7631104827 | 6211813364 | 4761640977 | =MMULT(MINVERSE(G2:K6),L2:L6) |                           |
| 7  | 66226872 | 636576.2 | 27679388 |             |            |           |            |            |            | (MMULT(array1,array2))        |                           |
| 8  | 28423480 | 2511442  | 46178212 |             |            |           |            |            |            | #N/A                          |                           |
| 9  | 51948515 | 730273.5 | 93125921 |             |            |           |            |            |            | #N/A                          |                           |
| 10 | 97431050 | 45979463 | 1.87E+08 |             |            |           |            |            |            | #N/A                          |                           |

通过上述网络训练，得到了确定的神经网络结构及其权值，该神经网络模型能很好的描述影响利润变化的因素与利润之间的映射关系，可根据影响利润的因素进行利润预测。

## 5.3 任务 3.3

### 5.3.1 财务造假识别方法

集成学习算法是通过构建并结合多个学习器来完成学习任务。包括但不限于 Adaboost 算法、GBDT 算法、XGBoost 算法和随机森林算法等等。

随机森林最重要的特点是随机性，它的随机性可以归因于样本随机与特征随机，并因此为学习器带来单一决策树所不具有的优势：通过随机生成样本进行学习的方式可以有效提高其泛化能力。因此，使用随机森林算法训练的模型识别财务报表造假具有较强的适配性，对于不同行业企业公司的造假行为都能保持较为稳定的识别准确率。但随机森林算法的缺点也较为明显，要提升模型识别准确率，不仅需要要求每个识别器提高学习的程度，还依赖抽样过程的质量，因此就需要设定更深的分裂程度，而大数据学习下识别模型的影响因素较多，如果要使每个识别器学习到更多的信息，会导致财务造假识别模型的训练时长急剧增加。

相比于随机森林算法，GBDT 算法提高模型识别准确率的速度更快，适合对特定行业进行高精度的判断，而 GBDT 存在问题是，该算法没有设计措施预防过拟合问题，但由于集成时每个弱识别器相互关联，会使得集成后的识别器对一类或几类报表指标具有判别上的倾向性，对财务造假的适用性略窄。因此，GBDT 算法可以对某一类财务造假识别具有高度的准确性，但在整个市场多个行业中使用该模型的错误率较高，因为该模型很可能对其他领域企业公司的财务造假不具有良好的识别能力。

XGBoost 算法作为一种高效的集成学习算法，其训练模型的逻辑是定义一个包含样本偏差与正则项的损失函数，使用 XGBoost 算法进行模型的训练，就是在有监督的条件下调试各个财务指标的参数，使其能准确识别实施财务造假的企业。XGBoost 算法之所以能够应用于财务造假识别领域，在于 XGBoost 算法能够生成准确率更高的财务造假识别模型。XGBoost 算法用于财务造假识别的具体训练过程如下：首先，XGBoost 算法会使用财务造假识别体系生成第一个弱学习器。进而，XGBoost 算法会在向前分布规则的指引下进行迭代，并与企业是否财务造假的真实结果进行比较，在每一次迭代过程中都寻找可以使该



差值下降最大的一个识别指标，为其赋予最合适的参数，并开始进行下一次迭代，逐步寻找多个可以有效识别财务造假的重要识别变量并为其赋值，最终使财务造假识别模型的准确率提升至可以应用于实际工作的水平。同时，利用二阶梯度缩小真实值与预测值之间的差距的方法，可以使损失函数收敛至设定条件的速度更快，从而使得模型预测的准确性提升至可以信赖的程度。XGBoost 算法使用二阶梯度拟合偏差，在设定的分裂深度下拥有最佳的训练速度与识别精度，相比于其他机器学习算法模型，使用 XGBoost 算法训练模型最快，可以迅速构建财务造假识别模型，并且 XGBoost 算法相比于其他机器学习算法能进一步提高识别的准确率。

本文中我们多方比较，最终选择使用 XGBoost 算法来建立财务造假的识别模型。

### 5.3.2 指标的初步选取与体系构建

结合财务造假的动因理论，从财务指标、压力因素、公司内部治理情况和外部因素这四个部分进行探索，本文从五大能力和风险状况这六个方面进行财务指标的选取：

针对上市公司财务造假措施的隐蔽性，则应结合造假企业常见的财务造假手段来选取指标。针对收入造假，可以从收入成本配比和收入异常增长中找到端倪，因此，选取应收账款周转率、应收账款变化率及存货周转率等运营指标可以发现其中的异常，并对企业是否实施财务造假有一个较好的判断。

基于舞弊风险因子模型，结合企业公司的现实情况，将动机定义为来自股东、债权人等的压力。其中，股东施加压力是因为公司发展不佳而选择撤资的情况，因此，本文使用资产报酬率盈利指标来衡量。债权人的压力则主要来源于破产压力，企业债务比例过高，势必会对企业经营的稳定性提出挑战，一旦企业的经营环境发生变化，高额债务就会迅速挤占企业的现金流，更有甚者，当企业无法偿还到期利息时，债权人有权申请该企业破产清算，因此，高额的债务与下滑的经营情况是管理层受债权人压力而进行财务造假的预兆，所以，本文选取流动比率盈利能力指标与资产负债率偿债能力指标衡量企业面临的债权人压力。

| 指标类型 | 变量名称与指标名称 | 变量定义及说明             |
|------|-----------|---------------------|
|      |           | 1=有财务造假行为，0=无财务造假行为 |
| 增长指标 | 流动比率      | 流动资产/流动负债           |
|      | 资产负债率     | 负债总额/资产总额           |
| 运营指标 | 存货周转率     | 主营业务成本/平均存货余额       |
|      | 应收账款周转率   | 主营业务收入/应收账款         |
| 利润指标 | 资产报酬率     | 息税前利润/总资产           |

在以上选取指标特征的基础上，对数据进行处理工作。

5.3.3 数据处理

在取得的初始样本集中，由于所选择的公司处于不同的行业，其特征指标资产规模、周转率等具有较大差别存在明显差异，因此为避免此类差距过大给最终结果带来影响，消除行业因素给特征变量造成的影响，对所有连续变量进行行业均值处理变换，将变量除以对应的行业均值，同时对部分缺失值采取行业均值填充，消除因行业不同导致特征差异过大的问题，避免行业因素的影响。

对于其中的缺失值，使用 XGBoost 算法不需要对缺失值进行处理，因为算法自带的稀疏性的离散特征可以在训练中完成缺失值的自动分配与填充。另外，鉴于 XGBoost 算法在处理高阶数据上的优异表现，无需考虑自变量过多会导致偏差过大的问题，因此，唯一可能影响模型效果的就是众多指标所带来的变量之间的相互关系，变量特征的高相关性，可能导致在计算信息增益时，模型会偏向于采用那些具有高度相关性的特征，过分依赖一种或几种相互关联的变量，会导致模型整体过拟合，因此，有必要进一步检验众多变量之间的相关关系，并对其中具有高度相关性的变量予以剔除，以保证最终模型的准确性。

关键因子的相关系数表：

### 5.3.4 模型设计

XGBoost 算法是 GBDT 算法在策略上的一种工程实现。该算法与 GBDT 一样，同样采用了梯度提升的运算框架。XGBoost 算法兼具了性能与速度，建立在缜密的数学逻辑上。首先，决策树可以被定义为：

$$\hat{y}_i = \sum_{t=1}^K f_t(x_i)$$

其中，K 是设置的弱学习器的个数， $x_i$  是代表输入的第 i 个样本， $\hat{y}$  代表在经过 k 次加法模型迭代后得出的模型预测值，这也是 Boosting 算法通用解决分类问题的策略。XGBoost 算法采用了前向分布算法逻辑，将包含 K 棵弱学习器按权重相加，并使用目标函数来定义优化的结果。

$$Obj(\theta) = \sum_{i=1}^N l(y_i, \hat{y}_i) + \sum_{j=1}^t \Omega(f_j)$$

此式中目标函数包含损失函数与正则项两部分组成，目标函数越小，则代表该模型的性能越好。损失函数衡量真实值与预测值之间的偏差，正则项部分则是用来限制决策树的复杂度，以提高模型整体的泛化能力。其中，正则项部分可以用下式来表示

$$\Omega(f_j) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

在上式中，树的叶子节点数 T 与惩罚系数  $\lambda$ 、每棵树叶子节点输出分数的平方和共同决定。其中，惩罚系数是人为设定的一个系数，取值越高，对泛化能力的要求就越高。从正则项的表达式来看，分裂次数增加，正则项的两个部分都会增加，因此，通过正则项可以很好的约束模型的过度拟合。

由于贪心算法的工作原理是在每一次迭代过程中生成一棵使损失函数降低

最大的树，因此将本次预测的结果改写为上一次的预测结果与本次决策树函数之和，此时可以用下式来表示

$$Obj = \sum_{i=1}^N l(y_i, \hat{y}_i^{t-1} + f_t(x_i)) + \Omega(f_t)$$

对于最优化问题，将目标函数使用二阶泰勒展开，使其逼近目标残差，可以得到

$$Obj = \sum_{i=1}^N (l(y_i, \hat{y}_i^{t-1}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)) + \Omega(f_t)$$

整理后，将输入的  $x$  映射到对应的叶子节点上：

$$f_t(x_i) = w_{q(x)}$$

由此，样本集由个体转为具体的叶节点：

$$I_j = \{i | q(x_i) = j\}$$

整理后的公式为：

$$Obj = \sum_{j=1}^T (G_j w_j + \frac{1}{2} w_j^2 (H_j + \lambda)) + \gamma T$$

最终，对叶节点输出分数  $w$  求导，即可得到第  $t$  轮中使得目标函数最小的输出分数，将此值带回目标函数，就可以反应当前叶子节点对总体损失降低的贡献，最终结果如下：

$$Obj^{(1)} = -\frac{1}{2} \sum_{j=1}^T \left( \frac{G_j^2}{H_j + \lambda} \right) + \gamma T$$

通过上述的运算步骤，XGBoost 就可以确定该节点的得分，最后，采取贪心算法或精确算法确定树的结构。

### 5.3.5 解决过程

Python

```
# 载入数据并作基本的数据预处理工作，方便后续回归预测的进行

data = pd.read_csv('financial_data.csv')

data['Month'] = data.YYYYMM.astype(str).str[4:6].astype(float)
data['Year'] = data.YYYYMM.astype(str).str[0:4].astype(float)
data.drop(['YYYYMM'], axis=1, inplace=True)

data.replace([np.inf, -np.inf], np.nan, inplace=True)

data.isnull().sum()

# 将月份和年份作为特征

X = data.loc[:, ['Month', 'Year']].values
y = data.loc[:, 'Value'].values

# 划分出训练集和测试集

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

首先，通过 pandas 库读取名为“financial\_data.csv”的数据集，并将日期列“YYYYMM”中的年份和月份提取出来，作为新的“Month”和“Year”两列特征，并删除原始的“YYYYMM”列。然后使用 numpy 库将数据集中的无穷大或无穷小值替换为 NaN（Not a Number）。统计数据集中每一列的缺失值数量，并输出结果。将提取出的特征变量“Month”和“Year”赋值给 X，将目标变量“Value”赋值给 y。接着，利用 scikit-learn 库中的 train\_test\_split 函数，将数据集划分为训练集（X\_train, y\_train）和测试集（X\_test, y\_test），其中测试集占总体数据的 20%，随机种子值设定为 42。这些处理过程为后续的回归预测模型训练和测试提供了基础数据，使得可以对数据进行进一步的分析和建模操作。

Python

```
reg_mod = xgb.XGBRegressor(  
    n_estimators=1000,  
    learning_rate=0.08,  
    subsample=0.75,  
    colsample_bytree=1,  
    max_depth=7,  
    gamma=0,  
)  
  
# 训练模型并指定评估数据集  
eval_set = [(X_train, y_train), (X_test, y_test)]  
  
reg_mod.fit(X_train, y_train, eval_set=eval_set, eval_metric='rmse',  
verbose=False)  
  
# 绘制损失曲线  
sns.set_style("white")
```

```

palette = sns.color_palette("Set2", n_colors=2)

plt.plot(reg_mod.evals_result()['validation_0']['rmse'],
label='train', color=palette[0], linewidth=2)

plt.plot(reg_mod.evals_result()['validation_1']['rmse'],
label='test', color=palette[1], linewidth=2)

plt.xlabel('Iteration')

plt.ylabel('RMSE')

plt.legend()

plt.savefig('Loss.png')

plt.show()

```

Python

```

from sklearn.metrics import r2_score

reg_mod.fit(X_train, y_train)

scores = cross_val_score(reg_mod, X_train, y_train, cv=10)

print("Mean cross-validation score: %.2f" % scores.mean())

rmse = np.sqrt(mean_squared_error(y_test, predictions))

print("RMSE: %f" % (rmse))

r2 = np.sqrt(r2_score(y_test, predictions))

print("R_Squared Score : %f" % (r2))

```

## 六、参考文献

[1]Messod D. Beneish.Detecting GAAP violation: implications for assessing earnings management among firms with extreme financial

performance[J]. Journal of Accounting and Public Policy, 1997(3).

[2] Bruno S. Frey. Yes, Managers Should Be Paid Like Bureaucrats[J]. Journal of Management Inquiry, 2005(1).

[3] 刘桂良; 叶宝松; 周兰;. 舞弊治理: 基于上市公司财务舞弊特征的分析[J]. 财经理论与实践, 2009(03).

[4] 陈慧璇; 朱君;. 我国上市公司财务报告舞弊特征分析[J]. 税务与经济, 2013(02).

[5] 卢馨; 李慧敏; 陈烁辉;. 高管背景特征与财务舞弊行为的研究——基于中国上市公司的经验数据[J]. 审计与经济研究, 2015(06).

[6] Lajos Zager;; Sanja Sever Malis;; Ana Novak. The Role and Responsibility of Auditors in Prevention and Detection of Fraudulent Financial Reporting[J]. Procedia Economics and Finance, 2016.

[7] Mark S. Beasley; Joseph V. Carcello; Dana R. Hermanson; Paul D. Lapedes. Fraudulent Financial Reporting: Consideration of Industry Traits and Corporate Governance Mechanisms[J]. Accounting Horizons, 2000(4).

[8] 阎达五; 王建英;. 上市公司利润操纵行为的财务指标特征研究[J]. 财务与会计, 2001(10).

[9] 王瑞雪; 乔瑞红;. 上市公司管理层舞弊动因及特征分析[J]. 财会通讯, 2015(33).

[10] 刘元; 林爱梅; 单雅迪;. 我国上市公司财务报告舞弊的特征和手段——基于 2008~2013 年证监会处罚公告[J]. 财会月刊, 2015(28).

[11] 黄世忠;. 上市公司财务造假的八因八策[J]. 财务与会计, 2019(16).

[12] Junli Huang; Jiapeng Gao ; Sheng Gao. Research on Important Risk Control Points of Enterprise Financial Management in the Process of IPO[J]. Frontiers in Economics and Management, 2021(8).

[13] 郭丽虹; 刘凤君;. 发审委关联、业绩粉饰与 IPO 审核决策[J]. 改



革, 2020(02).

[14]唐棱曦. 企业 IPO 财务舞弊动因及手段研究[D]. 云南财经大学, 2020(07).

[15]李春颖. IPO 前盈余管理的影响因素及经济后果[D]. 天津大学, 2018(07).

[16]贺小刚;连燕玲;吕斐斐;. 期望差距与企业家的风险决策偏好——基于中国家族上市公司的数据分析[J]. 管理科学学报, 2016(08).

[17]何威风;刘启亮;. 我国上市公司高管背景特征与财务重述行为研究[J]. 管理世界, 2010(07).

[18]顾亮;刘振杰;. 我国上市公司高管背景特征与公司治理违规行为研究[J]. 科学学与科学技术管理, 2013(02).

[19]吴丽君;卜华;. 民营企业高管背景与社会责任“迎合”行为研究[J]. 技术经济, 2020(11).

[20]武柢君. 上市公司高管薪酬水平与盈余管理关系研究[D]. 兰州大学, 2020(01).